Sistemas y tecnologÃas de administraciÃ³n del conocimiento

Knowledge Management System for Microsoft

Project progress report: lifecycle map, ontology, automated classification and semantic search

Student	Gael AndrÃ©s GarcÃa GarcÃa
Instructor	Jonathan Alexis Puente Guerrero
Course	Sistemas y tecnologÃas de administraciÃ³n del conocimiento
Deliverable	Project progress document
Date	10/30/2026
Repository	https://github.com/GaelGarcia0598/Microsoft-KM

1. Description of the company: Microsoft

Microsoft Corporation is a global technology company (headquartered in Redmond, Washington, USA) that develops software, cloud services (Azure, Microsoft 365, Dynamics 365), devices and AI products. It is the organization analysed in Activity 1. This project keeps the IT Technical Support domain of Activity 2 (customers, incidents, systems, technicians, tools and documents) and extends that ontology. *Note: the 48 sample documents are synthetic examples written in the style of Microsoft support, cloud, security and management documents; they are not real internal Microsoft documents.*

a) Type of organization

A multinational technology company (software, cloud and devices) that builds products and also operates and supports cloud infrastructure and business applications for consumers, enterprises and public institutions.

b) Core activities

Area	Core activities	Knowledge generated
Customer and technical support	Receives and resolves support cases by phone, email, chat and portal (24/7 for critical customers).	Troubleshooting solutions, recurring issues, customer feedback.
Cloud and infrastructure operations	Operates Azure, networks, VPN gateways, Exchange Online, backups and customer cloud migrations.	Incident reports, configurations, root-cause analyses.
Security	Microsoft Defender, Microsoft Entra ID access control, audits, phishing response.	Policies, audit findings, threat lessons learned.
Product engineering and projects	Development on GitHub and Azure DevOps, platform upgrades, customer onboarding.	Technical designs, progress and closure reports.
Management	SLA monitoring, planning, training (Microsoft Learn), customer relationship.	Performance reports, KPIs, satisfaction surveys.

c) Types of knowledge and documents handled

Microsoft manages both **explicit** knowledge (documents) and **tacit** knowledge (the diagnostic judgment of senior engineers, which the lifecycle tries to turn into documents). Six document types are handled and are the categories used by the classifier:

Document type (category)	Purpose	Typical content
Policy (policy)	Rules and obligations	Password, remote work, change management, backup, escalation.
Manual (manual)	How-to instructions for users	Step-by-step guides: VPN client, email on mobile, printer setup.
Incident report (incident_report)	Record of what happened	Timeline, affected system, root cause, corrective action.
KB article (kb_article)	Fast troubleshooting	Symptom, cause, solution; e.g. VPN error 809.
Technical documentation (technical_doc)	How systems are built	Architecture, API reference, schema, configuration.
Project / performance report (project_report)	Management information	KPIs, SLA compliance, audits, project status.

2. Knowledge lifecycle map

The map extends the five stages of Activity 1 (Microsoft) to six stages adapted to Microsoft, separating *organize / classify* from *store / document* as required by this project. Gold stages are the ones supported by the Python prototype.

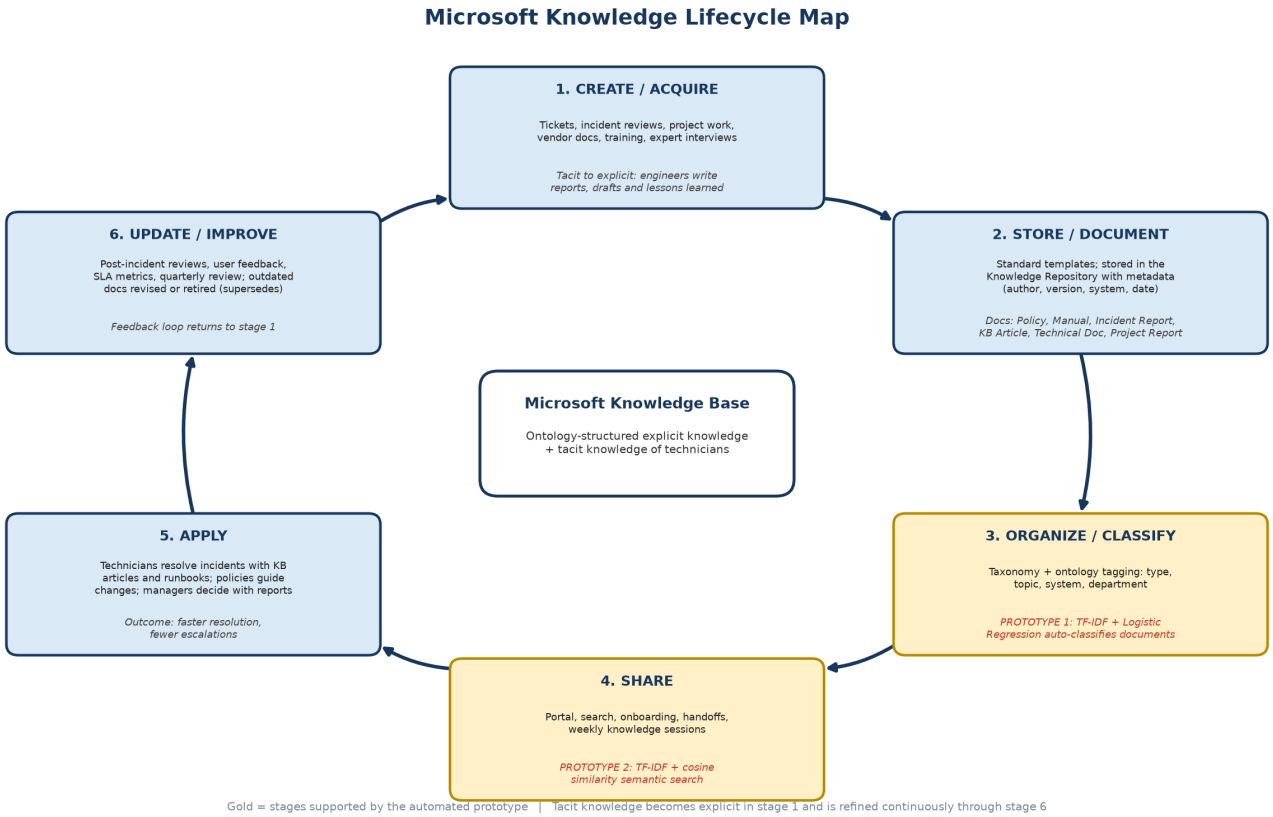


Figure 1. Knowledge lifecycle map of Microsoft.

Stage	How it happens at Microsoft	Documents / outputs	Support
1. Creation / acquisition	Technicians capture lessons from tickets and incident reviews; project teams write designs; training and vendor documentation is acquired.	Incident reports, drafts, designs	Templates
2. Storage / documentation	Documents are saved in the repository using standard templates and metadata (author, version, system, date).	All six document types	Repository
3. Organization / classification	Each document is tagged by type, topic, system and department following the ontology.	Metadata, category labels	**Prototype 1: classifier**
4. Sharing	Knowledge portal and search, onboarding, handoffs and weekly knowledge sessions.	Search results, training	**Prototype 2: search**
5. Application	Technicians resolve incidents with KB articles; policies guide changes; managers use reports to decide.	Resolved tickets, decisions	-
6. Update / improvement	Post-incident reviews, feedback and SLA metrics trigger revision; old documents are superseded or retired.	New versions, lessons learned	Review cycle

3. Ontology / conceptual graph of organizational knowledge

The ontology (`ontology/MicrosoftKM.owl`, OWL/RDF-XML) extends the Activity 2 model (Incident, Technician, System, Tool, Document, Customer) with the classes needed to organize knowledge. It has **15 classes**, **20 object properties**, 8 data properties and sample individuals (e.g. incident INC-1001, KB-201, a remote-work policy).

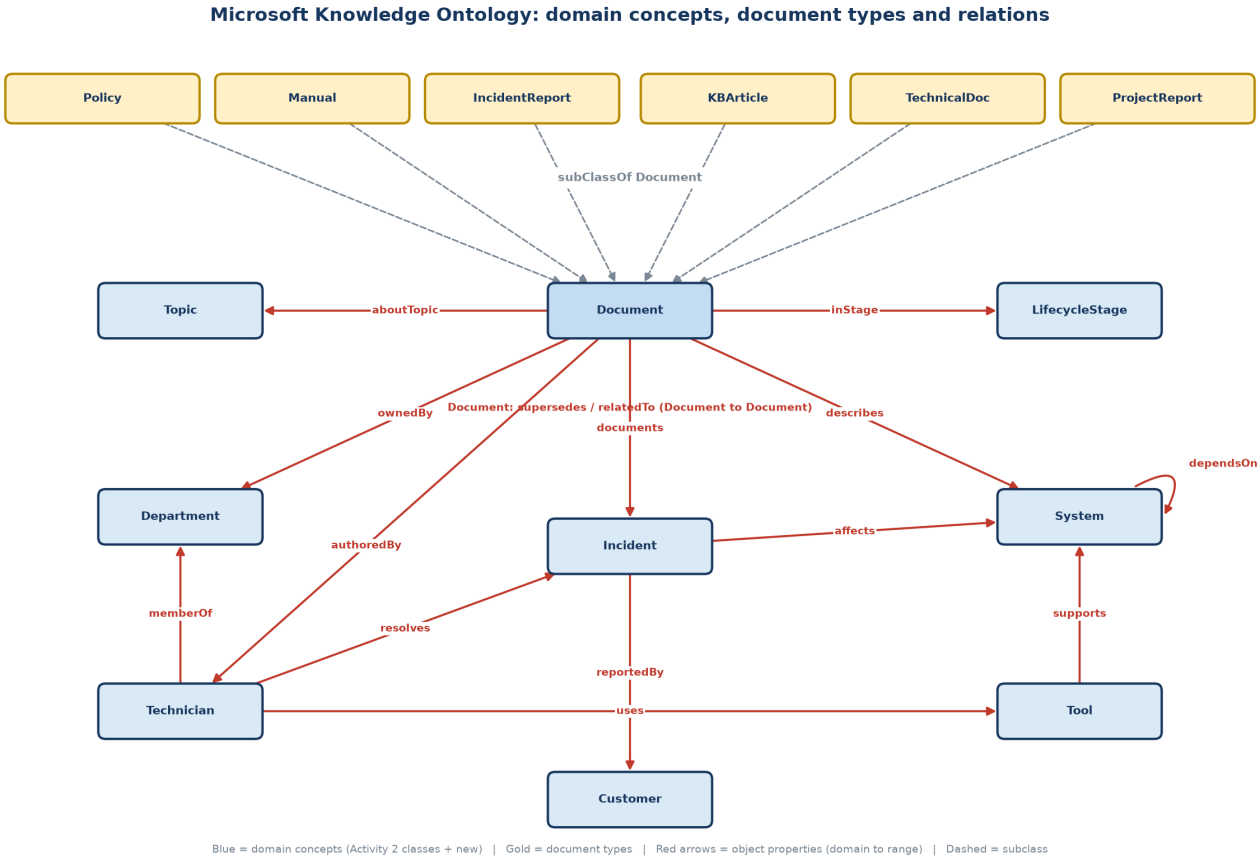


Figure 2. Ontology graph: domain concepts (blue), document types (gold) and relationships (red arrows).

Elements of the model

Element	Content
a) Main concepts	Incident, System, Technician, Tool, Customer, Topic, Department, LifecycleStage, Document.
b) Document types	Policy, Manual, IncidentReport, KBArticle, TechnicalDoc, ProjectReport (all subclasses of Document).
c) Relations between concepts	reportedBy, affects, resolves, uses, supports, dependsOn, memberOf.
d) Relations between documents and concepts	documents, describes, aboutTopic, authoredBy, ownedBy, inStage, governs, guides, solves, summarizes; document to document: derivedFrom, supersedes, relatedTo.

Document type to concept relation matrix

	System	Customer	Incident	Topic	Technician	Department	Document
Policy	governs			aboutTopic			
Manual		guides		aboutTopic			
IncidentReport	describes		documents		authoredBy		
KBArticle	describes		solves	aboutTopic			derivedFrom IncidentReport
TechnicalDoc	describes						relatedTo / supersedes
ProjectReport			summarizes		authoredBy	ownedBy	

Figure 3. Which document types relate to which concepts, and through which property.

Example of how knowledge is connected: incident **INC-1001** (VPN outage) *affects* the Azure VPN Gateway and is *resolved by* a support engineer. Its **incident report** *documents* the incident; the **KB-201** article is *derivedFrom* that report, *solves* the incident, and *describes* the Azure VPN Gateway; the **remote-work policy** *governs* the same system. A user searching for "VPN" can therefore reach the policy, the manual, the incident and the fix through the graph.

4. Explanation of the prototype

The prototype is written in Python 3 with scikit-learn and has two components that implement the stages *organize / classify* and *share* of the lifecycle. Source code and test documents are in the repository.

File	Role
build_corpus.py	Creates the 48 sample documents (data/documents/<category>/*.txt).
src/classifier.py	TF-IDF + Naive Bayes / Logistic Regression classifier, evaluation and prediction of new text.
src/semantic_search.py	TF-IDF vectors, cosine similarity and ranked results; optional ontology-based query expansion.
src/evaluate_search.py	12-query evaluation of the search component.
src/build_diagrams.py	Generates the OWL file, ontology graphs and lifecycle map.
ontology/*.owl	MicrosoftKM.owl (this project) and ITSupportOntology_Activity2.owl (previous activity).

4.1 Document set

48 short documents, 8 per category (policy, manual, incident_report, kb_article, technical_doc, project_report), written to resemble Microsoft support, cloud, security and management documents (synthetic, not real internal documents). The first line is the title and the rest is the body.

4.2 Automated classification

Steps: (i) load all documents and their category labels; (ii) vectorize with `TfidfVectorizer` (lowercase, English stop words, unigrams + bigrams, sublinear term frequency); (iii) split 75% / 25% with stratification (36 train / 12 test) and train Multinomial Naive Bayes and Logistic Regression; (iv) compare using test accuracy and 5-fold cross-validation, keep the best model and predict the category (with probability) of new documents.

```
vec = TfidfVectorizer(stop_words='english', ngram_range=(1, 2), sublinear_tf=True)
model = make_pipeline(vec, LogisticRegression(max_iter=1000, C=10))
model.fit(X_train, y_train)
model.predict_proba([new_text])      # probability per category
```

4.3 Semantic search

Steps: (i) the user types a natural language query; (ii) the query is transformed with the same fitted TF-IDF vectorizer into a vector; (iii) cosine similarity between the query vector and every document vector is computed; (iv) the top-k documents with score > 0 are shown with their category and similarity. An optional flag `--expand` adds related terms taken from the ontology concepts (e.g. *slow* adds *performance, freeze, disk usage*) so that the search is less dependent on exact words.

```
qvec   = vectorizer.transform([query])
scores = cosine_similarity(qvec, doc_matrix)[0]
top    = scores.argsort()[::-1][:k]
```

Limitation: TF-IDF matches shared vocabulary; it does not understand meaning. That is why it is called *basic* semantic search. Synonyms that are not in the ontology lexicon are not captured, and the document set is small and synthetic, so the results demonstrate the method, not production accuracy.

5. Evidence of prototype operation

The following captures show the real console output of the scripts (rendered as terminal images by `src/make_screenshots.py` from the actual execution results).

5.1 Classification

```
PS Microsoft-KM> python src/classifier.py   (training and evaluation)


Documents loaded: 48  |  train: 36  test: 12
Categories: policy, manual, incident_report, kb_article, technical_doc, project_report

=== naive_bayes ===
Test accuracy: 83.33%   5-fold CV: 87.56% (+/- 7.99%)
                precision    recall  f1-score   support

        policy       1.00      1.00      1.00         2
        manual       0.67      1.00      0.80         2
incident_report      0.50      0.50      0.50         2
     kb_article      1.00      1.00      1.00         2
  technical_doc      1.00      0.50      0.67         2
 project_report      1.00      1.00      1.00         2

       accuracy                           0.83        12
      macro avg       0.86      0.83      0.83        12
   weighted avg       0.86      0.83      0.83        12

=== logistic_regression ===
Test accuracy: 91.67%   5-fold CV: 93.78% (+/- 8.12%)
                precision    recall  f1-score   support

        policy       1.00      1.00      1.00         2
        manual       1.00      1.00      1.00         2
incident_report      0.67      1.00      0.80         2
     kb_article      1.00      1.00      1.00         2
  technical_doc      1.00      0.50      0.67         2
 project_report      1.00      1.00      1.00         2

       accuracy                           0.92        12
      macro avg       0.94      0.92      0.91        12
   weighted avg       0.94      0.92      0.91        12

Selected model: logistic_regression
```

Figure 4. Training and evaluation of both models.

On the 12-document test set Naive Bayes reached 83.3% and Logistic Regression 91.7%; with 5-fold cross-validation Naive Bayes obtained 87.6% and Logistic Regression 93.8%, so Logistic Regression was selected. The test set is small, so the cross-validation figure is the more reliable estimate.

```
PS Microsoft-KM> python src/classifier.py   (test predictions and new documents)


Test-set predictions:
  [OK ] manual_03           true=manual           pred=manual
  [OK ] kb_article_02       true=kb_article       pred=kb_article
  [ERR] technical_doc_04    true=technical_doc    pred=incident_report
  [OK ] incident_report_04  true=incident_report  pred=incident_report
  [OK ] project_report_04   true=project_report   pred=project_report
  [OK ] policy_02           true=policy           pred=policy
  [OK ] policy_04           true=policy           pred=policy
  [OK ] manual_08           true=manual           pred=manual
  [OK ] kb_article_07       true=kb_article       pred=kb_article
  [OK ] technical_doc_02    true=technical_doc    pred=technical_doc
  [OK ] project_report_05   true=project_report   pred=project_report
  [OK ] incident_report_07  true=incident_report  pred=incident_report

Confusion matrix saved to C:\Users\gaela\NovaTech-KM\evidence\confusion_matrix.png

Classification of NEW unseen documents:

  Text: Step 1: open Settings. Step 2: click Add device and follow the wizard to install the scann...
  -> Predicted category: manual (62%)  | 2nd: technical_doc (8%)

  Text: The mail server stopped responding at 03:20. The on-call engineer restarted it and the roo...
  -> Predicted category: incident_report (39%)  | 2nd: kb_article (15%)

  Text: Employees must not share their credentials. Failure to comply will result in sanctions.
  -> Predicted category: policy (21%)  | 2nd: kb_article (20%)

  Text: Symptom: the laptop cannot connect to Wi-Fi. Cause: outdated driver. Solution: reinstall t...
  -> Predicted category: kb_article (40%)  | 2nd: incident_report (15%)

  Text: The system architecture includes an API gateway, three microservices and a PostgreSQL data...
  -> Predicted category: technical_doc (39%)  | 2nd: incident_report (14%)

  Text: Quarterly report: customer satisfaction rose 8 percent and the project stayed within budge...
  -> Predicted category: project_report (61%)  | 2nd: incident_report (9%)
```

Figure 5. Test predictions and classification of six new, unseen texts.

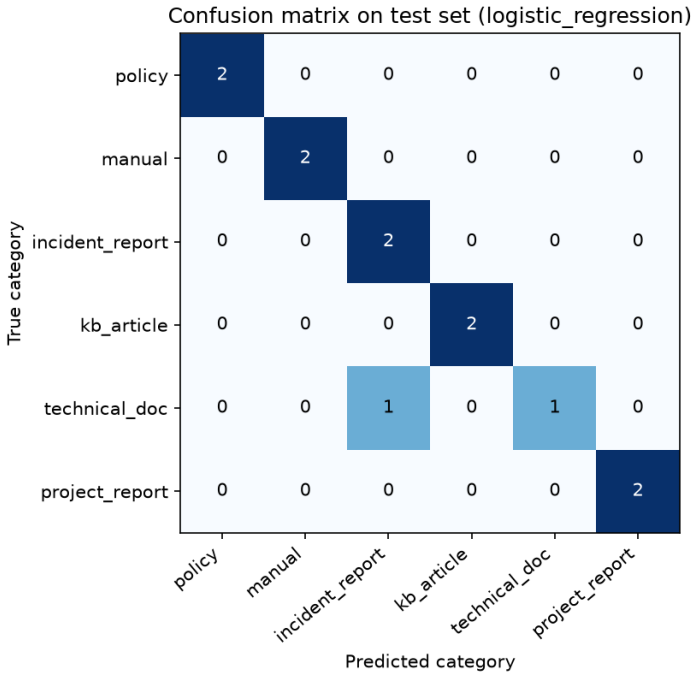


Figure 6. Confusion matrix of the selected model on the test set.

5.2 Semantic search

```
PS Microsoft-KM> python src/semantic_search.py "how do I fix a VPN connection problem"


Index built: 48 documents, 2113 TF-IDF terms

Query: "how do I fix a VPN connection problem"
  1. [0.139] (technical_doc) Technical Specification: VPN Gateway Architecture
  2. [0.129] (kb_article) KB-252: Printer Offline Error
  3. [0.110] (kb_article) KB-201: Troubleshooting VPN Error 809
  4. [0.095] (kb_article) KB-214: How to Fix Outlook Not Syncing
  5. [0.072] (manual) User Manual: Configuring the Azure VPN Client
```

Figure 7. Query: how do I fix a VPN connection problem.

The top result is the VPN technical specification; the VPN KB article (the most useful answer) ranks third, behind the printer-offline KB article that matched the word "fix". This is a real weakness of plain TF-IDF and is reported in the evaluation below.

```
PS Microsoft-KM> python src/semantic_search.py "steps to install a printer"


Index built: 48 documents, 2113 TF-IDF terms

Query: "steps to install a printer"
  1. [0.217] (manual) Operating Manual: Network Printer Setup
  2. [0.148] (kb_article) KB-252: Printer Offline Error
  3. [0.069] (policy) Acceptable Use Policy
  4. [0.067] (manual) Manual: How to Request a New Employee Account
  5. [0.066] (manual) Manual: Conference Room Equipment Guide
```

Figure 8. Query: steps to install a printer.

```
PS Microsoft-KM> python src/semantic_search.py   (demo, with and without ontology expansion)



Query: "my laptop is very slow"
  1. [0.255] (kb_article) KB-230: Resolving Slow Computer Performance
  2. [0.080] (manual) Quick Start Guide: Onboarding a Laptop to Microsoft Defender for Endpoint
  3. [0.053] (manual) Manual: Conference Room Equipment Guide
  4. [0.052] (incident_report) Incident Report INC-1031: Ransomware Attempt Blocked

Query: "my laptop is very slow"  [ontology expansion ON]
  1. [0.482] (kb_article) KB-230: Resolving Slow Computer Performance
  2. [0.060] (project_report) Quarterly Performance Report Q2
  3. [0.043] (manual) Quick Start Guide: Onboarding a Laptop to Microsoft Defender for Endpoint
  4. [0.033] (incident_report) Incident Report INC-1023: Database Server Crash
  5. [0.028] (manual) Manual: Conference Room Equipment Guide
```

Figure 9. Effect of the ontology-based query expansion (score of the best match rises from 0.255 to 0.482).

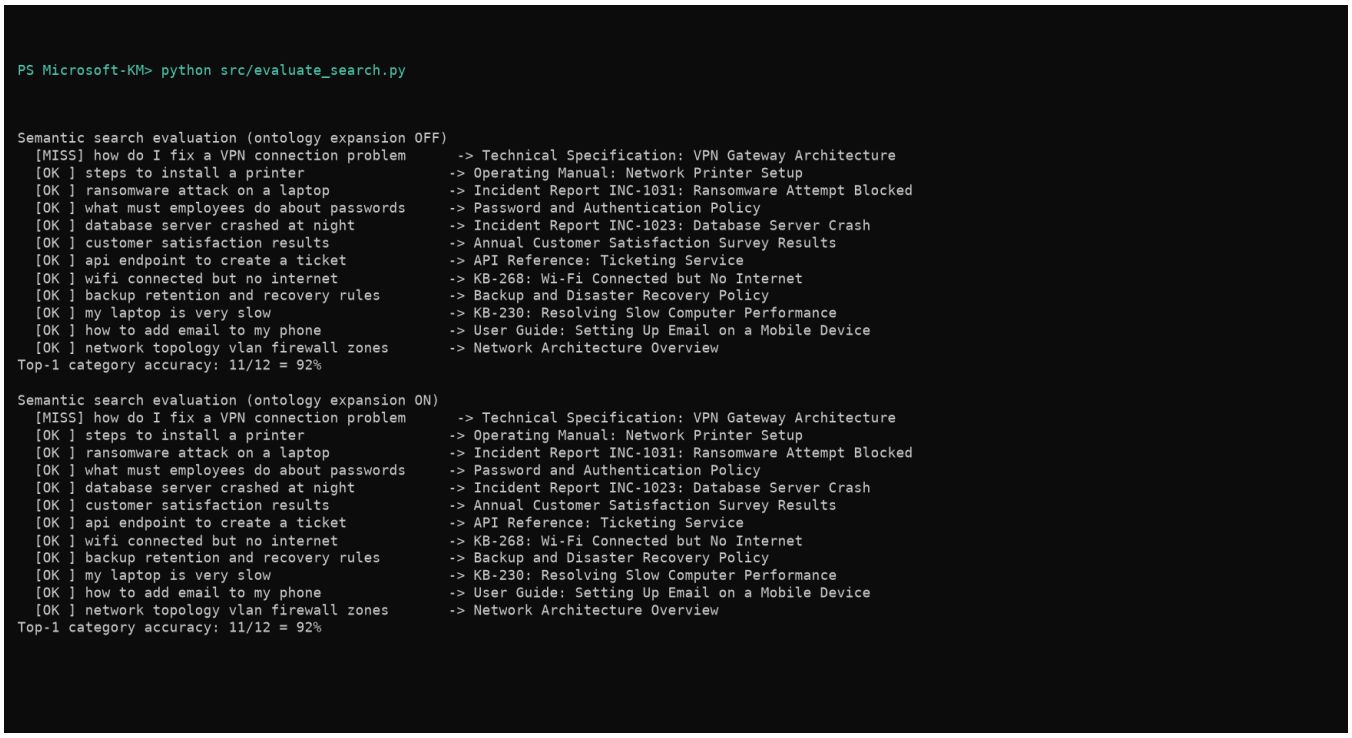


Figure 10. Evaluation with 12 queries, with and without expansion.

The search returned a document of the expected category as first result in **11 of 12 queries (92%)**. The only miss is the VPN query above. Expansion did not change the top-1 accuracy on this set but increased similarity scores for matching documents.

6. Evidence of earlier practical exercises

The project builds directly on the previous activities and on the experiments carried out while developing the prototype.

Exercise	What was done	Contribution to this project
Activity 1: Knowledge life cycle at Microsoft	Analysis of knowledge creation, storage, sharing, application and update; tacit vs explicit knowledge; AI support (classification, semantic search, outdated-content detection).	Base of the lifecycle map (Section 2) and the idea of AI-assisted classification and search.
Activity 2: IT Technical Support ontology	OWL ontology with 6 classes, 9 object properties and 8 data properties, checked with the HermiT reasoner.	Extended into MicrosoftKM.owl (Section 3), keeping its classes and properties.
Classification tests	Naive Bayes vs Logistic Regression, hold-out and 5-fold cross-validation, confusion matrix, prediction of new documents.	Prototype 1 (Section 5.1).
Information retrieval exercises	TF-IDF vectors, cosine similarity ranking, query expansion with ontology terms, 12-query evaluation.	Prototype 2 (Section 5.2).
Conceptual representation	Ontology graph, document-concept relation matrix and lifecycle map.	Figures 1 to 3.

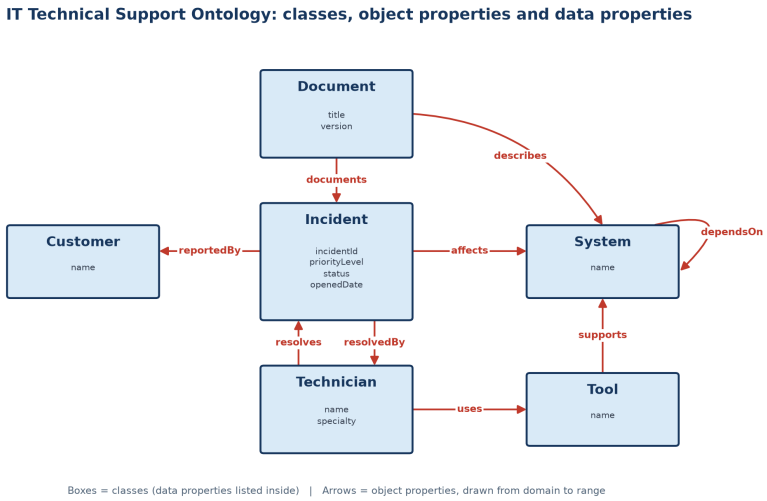


Figure 11. Ontology graph from Activity 2 (starting point of the new ontology).

Activity 1 and Activity 2 reports are the submitted evidence of those exercises.

7. Conclusions and next steps

The prototype shows that classification and retrieval can support two lifecycle stages with very little infrastructure. Next steps: expand the corpus with real, anonymized documents; add metadata extraction (system, topic) so documents are tagged with ontology individuals; replace TF-IDF with embeddings for true semantic matching; and add a review step where a person confirms the predicted category, consistent with the human-review principle of Activity 1.

Repository: https://github.com/GaelGarcia0598/Microsoft-KM